

Table of Contents

Unit 4: Computational Thinking	5
Student Learning Outcomes	5
Introduction	5
4.1 Computational Thinking	6
What Is Computational Thinking?	6
4.1.1 Decomposition	6
The Hook	6
The Explanation	7
Interactive Stop-Point: Pause & Think	7
Quick Recap	8
4.1.2 Pattern Recognition	8
The Hook	8
The Explanation	8
Class Activity	9
Interactive Stop-Point: Grab a Partner	9
Quick Recap	9
4.1.3 Abstraction	9
The Hook	10
The Explanation	10
Interactive Stop-Point: Pause & Think	10
Quick Recap	11
4.1.4 Algorithms	11
The Hook	11
The Explanation	11
Interactive Stop-Point: Grab a Partner	12
Quick Recap	12
4.2 Principles of Computational Thinking	13
Overview	13
4.2.1 Problem Understanding	13
The Hook	13
The Explanation	13
Interactive Stop-Point: Pause & Think	14
Quick Recap	14
4.2.2 Problem Simplification	14
The Hook	14

The Explanation	15
Interactive Stop-Point: Grab a Partner	15
Quick Recap.....	15
4.2.3 Solution Selection and Design.....	15
The Hook.....	16
The Explanation	16
Interactive Stop-Point: Pause & Think	16
Quick Recap.....	17
4.3 Algorithm Design Methods.....	17
Introduction	17
4.3.1 Flowcharts	17
The Hook.....	17
Importance of Flowcharts.....	17
Flowchart Symbols.....	18
Flowchart Examples.....	18
Class Activity	20
Interactive Stop-Point: Pause & Think	20
Quick Recap.....	20
4.3.2 Pseudocode.....	21
The Hook.....	21
The Explanation	21
Pseudocode Examples	21
Class Activity	23
Interactive Stop-Point: Pause & Think	23
Quick Recap.....	24
4.3.3 Differentiating Flowcharts and Pseudocode	24
Interactive Stop-Point: Grab a Partner	25
Quick Recap.....	25
4.4 Evaluation Techniques for an Algorithm	26
Introduction	26
4.4.1 Time Complexity	26
The Hook.....	26
The Explanation	26
Class Activity	27
Interactive Stop-Point: Pause & Think	27
Quick Recap.....	28
4.4.2 Space Complexity.....	28
The Hook.....	28

The Explanation	28
Interactive Stop-Point: Pause & Think	29
Quick Recap.....	29
4.5 Dry Run	29
Introduction	29
4.5.1 Dry Run of a Flowchart.....	29
The Hook.....	29
Step-by-Step Walkthrough	30
Class Activity	30
Interactive Stop-Point: Pause & Think	30
Quick Recap.....	31
4.5.2 Dry Run of Pseudocode	31
The Hook.....	31
Step-by-Step Walkthrough	31
Interactive Stop-Point: Grab a Partner	32
Quick Recap.....	32
4.5.3 Simulation	32
The Hook.....	33
The Explanation	33
Class Activity: Simulation Game	33
Interactive Stop-Point: Pause & Think	34
Quick Recap.....	34
4.6 Introduction to LARP (Logic of Algorithms for Resolution of Problems)	34
What Is LARP?	34
4.6.1 Why Is LARP Important?	35
The Hook.....	35
The Explanation	35
Interactive Stop-Point: Pause & Think	35
Quick Recap.....	35
4.6.2 Writing Algorithms in LARP.....	35
LARP Syntax — The Commands.....	35
LARP Algorithm Examples	36
Interactive Stop-Point: Grab a Partner	37
Quick Recap.....	37
4.6.3 Drawing Flowcharts in LARP	37
The Explanation	37
Class Activity	38
Interactive Stop-Point: Pause & Think	38

Quick Recap.....	39
4.7 Error Identification and Debugging	39
Introduction	39
4.7.1 Types of Errors	39
Error Type 1: Syntax Errors	39
Error Type 2: Runtime Errors.....	40
Error Type 3: Logical Errors.....	41
Summary Table: Types of Errors	42
Class Activity	42
Interactive Stop-Point: Pause & Think	42
Quick Recap.....	43
4.7.2 Common Error Messages in LARP	43
Debugging Strategy: The 4-Step Method.....	43
Interactive Stop-Point: Grab a Partner	44
Quick Recap.....	44
Chapter Summary	44
Key Vocabulary.....	46
Review Questions.....	46

Unit 4: Computational Thinking

"Everyone in this country should learn to program a computer, because it teaches you how to think."

— Steve Jobs

Student Learning Outcomes

By the end of this chapter, you will be able to:

- Define **computational thinking** and its four key components: decomposition, pattern recognition, abstraction, and algorithms.
 - Explain the **principles of computational thinking**: problem understanding, problem simplification, and solution selection and design.
 - Describe two **algorithm design methods** — flowcharts and pseudocode — and explain the differences between them.
 - **Create and interpret flowcharts** to represent algorithms visually.
 - **Write pseudocode** to outline algorithms in a structured, human-readable format.
 - Perform **dry runs** of flowcharts and pseudocode to manually verify their correctness.
 - Understand the concept and purpose of **LARP** (Logic of Algorithms for Resolution of Problems).
 - **Identify** three types of errors in algorithms: syntax errors, logical errors, and runtime errors.
 - Apply **debugging techniques** to find and fix errors.
 - Demonstrate **problem-solving skills** by applying computational thinking to real-world scenarios.
-

Introduction

Imagine you have never baked a cake before. You walk into a kitchen and there is flour, sugar, eggs, butter, and an oven. Can you just start mixing things randomly? Probably not. You would look for a **recipe** — a clear, step-by-step guide that tells you exactly what to do and in what order.

That recipe is an algorithm. And the process of reading the recipe, gathering the right ingredients, and thinking through the steps before you start? That is **computational thinking**.

Computational thinking is not about computers. It is a **way of thinking**. It is a set of problem-solving skills that helps you break big, scary problems into small, manageable pieces — and then solve them, one step at a time.

You already use computational thinking in daily life. When you plan a route to school, when you organize your school bag, when you figure out how many minutes until lunch — you are thinking computationally.

In this chapter, you will learn to use these skills **consciously and systematically**. You will learn to draw flowcharts, write pseudocode, test your logic, and fix mistakes. By the end, you will see the world a little differently — not just as a student, but as a **problem-solver**.

Let's begin.

4.1 Computational Thinking

What Is Computational Thinking?

Definition: Computational thinking (CT) is a problem-solving process. It involves a set of skills and techniques to solve complex problems in a way that a computer — or any logical system — can understand and execute.

The important point: **you do not need a computer to think computationally**. Scientists, doctors, architects, and teachers all use computational thinking. It is a universal skill.

Computational thinking has four main components:

1. **Decomposition**
2. **Pattern Recognition**
3. **Abstraction**
4. **Algorithms**

Let's explore each one.

4.1.1 Decomposition

The Hook

It is the first day of summer break. Your mother asks you to **clean your entire room**. You look at it — clothes on the floor, books piled up, a dusty desk, shoes everywhere — and you feel completely overwhelmed. Where do you even start?

But then you think: what if you do **one thing at a time**?

1. First, pick up all the clothes and put them in the laundry basket.
2. Then, stack the books on the shelf.
3. Then, wipe the desk.
4. Then, arrange the shoes.
5. Finally, sweep the floor.

Suddenly, a huge, scary task becomes five small, easy tasks. **That is decomposition.**

The Explanation

Definition: Decomposition is the method of breaking down a complicated problem into smaller, more convenient, and manageable components.

When a problem is large, it is hard to know where to begin. Decomposition gives us a starting point. It helps us see the individual parts that make up the whole.

Example: Building a Birdhouse

Imagine you want to build a birdhouse. The full task sounds complicated. But watch what happens when we decompose it:

Step	Sub-Task
1	Design the birdhouse — decide size, shape, and sketch a plan.
2	Gather materials — wood, nails, paint, hammer, saw.
3	Cut the wood — measure and cut pieces according to the design.
4	Assemble the pieces — join the wood pieces to form the structure.
5	Paint and decorate — make it attractive for birds.
6	Install the birdhouse — find a good location and fix it securely.

Each step is simple on its own. Together, they build a birdhouse. Decomposition made the impossible feel possible.

Interactive Stop-Point: Pause & Think

Your Turn: You want to organize a **class science exhibition**. It feels huge. Decompose it.

Write down at least **5 smaller sub-tasks** that together make up the full task of organizing the exhibition.

(Example starter: "1. Choose a topic for each group.")

Can you add 4 more? Discuss with your partner.

Quick Recap

Decomposition = Breaking one big problem into many small, solvable steps.

4.1.2 Pattern Recognition

The Hook

Your mathematics teacher gives you a sequence: **1, 4, 9, 16, 25 ...**

"What comes next?" she asks.

You stare. Then you notice something. $1 = 1 \times 1$. $4 = 2 \times 2$. $9 = 3 \times 3$. $16 = 4 \times 4$. $25 = 5 \times 5$.

They are all **perfect squares**! So the next number must be $6 \times 6 = \mathbf{36}$.

You did not memorize that answer. You found a **pattern**. And once you found it, you could predict any number in the sequence. That is pattern recognition.

The Explanation

Definition: Pattern recognition is the process of identifying regularities, similarities, or repeating structures within a set of data or problems.

When we recognize a pattern, we can:

- **Predict** what comes next.
- **Reuse** a solution we already know.
- **Save time** instead of solving from scratch every time.

Example: Areas of Squares

Look at how the area of a square grows as the side length increases:

Side Length	Area	How It's Built
1	1	1
2	4	1 + 3
3	9	1 + 3 + 5
4	16	1 + 3 + 5 + 7
5	25	1 + 3 + 5 + 7 + 9

The pattern: Each new area is formed by adding the next **consecutive odd number**.

Once you see this pattern, you do not need to square the number every time. You can just add the next odd number. This is pattern recognition saving you work.

Class Activity

Create a table with side lengths from **1 to 10**. Use the pattern (adding consecutive odd numbers) to fill in the area for each. Then verify your answers by squaring the side lengths. Do they match? They should!

Interactive Stop-Point: Grab a Partner

Look at this sequence of numbers: **2, 6, 18, 54, 162 ...**

With your partner, answer these questions:

1. What is the pattern? (Hint: what are you multiplying by each time?)
2. What is the **6th** number in the sequence?
3. What is the **10th** number?

Quick Recap

Pattern Recognition = Spotting a repeating rule in data so you can predict, reuse, and save effort.

4.1.3 Abstraction

The Hook

You are using Google Maps to find your school. The map shows roads, labels, and a blue dot for your location. It does **not** show every tree, every building's interior, every person walking on the road.

Why not? Because you do not need that information to find your school. The map hides the unnecessary details and shows you only what matters. **That is abstraction.**

The Explanation

Definition: Abstraction is the process of hiding complex details and showing only the necessary information. It helps reduce complexity so we can focus on what is important.

Abstraction is everywhere:

- A **map** abstracts reality — it shows roads, not every tree.
- A **recipe** abstracts chemistry — it says "bake at 180°C" without explaining combustion.
- A **phone icon** on your screen abstracts millions of lines of code — you just tap it.

Example: Making a Cup of Tea

The full chemistry of making tea involves boiling water (which means exciting water molecules), the extraction of tannins and caffeine through diffusion... and so on.

But the **abstracted version** is clean and simple:

1. Boil water.
2. Add a tea bag.
3. Wait 3 minutes.
4. Remove the bag. Add milk or sugar if you like.
5. Drink.

You did not need to understand chemistry. The abstraction gave you exactly what you needed.

In computing, abstraction means we design systems in layers. A programmer writing a game does not worry about how the computer stores data in binary. Those details are **abstracted away** by the operating system and programming language.

Interactive Stop-Point: Pause & Think

Think about **driving a car**.

A driver uses: the steering wheel, the accelerator, the brakes, and the gear shift.

A driver does NOT need to know: how the engine converts fuel into motion, how the transmission system works, or how the brake pads create friction.

Question: What details has the car **abstracted away** from the driver? Can you list three more examples of abstraction in daily life?

Quick Recap

Abstraction = Hide what is not needed. Show only what matters. Focus on the big picture.

4.1.4 Algorithms

The Hook

In 1843, a mathematician named **Ada Lovelace** wrote a set of step-by-step instructions for a mechanical computing machine built by Charles Babbage. It was the very first algorithm written for a machine. Ada Lovelace is considered the world's first computer programmer — and she never even saw a modern computer.

Her insight? That a machine could follow instructions if those instructions were clear, ordered, and complete. A good algorithm is exactly that: **clear, ordered, and complete**.

The Explanation

Definition: An algorithm is a step-by-step set of instructions used to solve a problem or complete a task.

Think of an algorithm like a **recipe**, a set of **driving directions**, or a **game rulebook**. Each one tells you exactly what to do, in what order, to achieve a specific result.

Key Properties of a Good Algorithm:

Property	Meaning
Clear	Each step is specific. No guessing.
Ordered	Steps are in the correct sequence.
Finite	The algorithm must eventually end.
Effective	It must actually solve the problem.

Example: Planting a Tree

Here is a simple, real-life algorithm:

1. Choose a suitable spot in your garden.
2. Dig a hole twice the width of the tree's root ball.
3. Place the tree in the hole. Make sure it is upright.
4. Fill the hole with soil. Press gently to remove air pockets.
5. Water the tree generously.
6. Add mulch around the base to retain moisture.
7. Water the tree regularly until it is established.

Notice: each step is specific. The order matters (you cannot fill the hole before placing the tree). And the algorithm has a clear end.

Example: Adding Two Numbers (Computer Algorithm)

1. START
2. Read the first number. Store it as A.
3. Read the second number. Store it as B.
4. Calculate $\text{Sum} = A + B$.
5. Display Sum.
6. STOP

Interactive Stop-Point: Grab a Partner

Write an algorithm for **making a cheese sandwich**. Your algorithm must have at least 6 steps. Be specific — imagine you are writing it for someone who has never made a sandwich before. Exchange your algorithm with another pair. Can they follow it exactly? Does it work?

Quick Recap

An algorithm is a clear, ordered, finite set of instructions that solves a specific problem.

4.2 Principles of Computational Thinking

Overview

Computational thinking is not just about the four components above. It also involves a **process** — a way of approaching any problem systematically. This process has three key principles:

1. **Problem Understanding**
 2. **Problem Simplification**
 3. **Solution Selection and Design**
-

4.2.1 Problem Understanding

The Hook

Albert Einstein once said:

"If I had an hour to solve a problem, I'd spend 55 minutes thinking about the problem and 5 minutes thinking about solutions."

Why? Because if you misunderstand the problem, your solution — no matter how brilliant — will be **wrong**. The most common mistake students make in exams is misreading the question. In computer science, the same mistake in a program can cause a complete system failure.

Understanding the problem is the most important step.

The Explanation

Definition: Problem Understanding means thoroughly analyzing a problem to identify its **core issue**, its **requirements**, and its **goals** — **before** attempting any solution.

To understand a problem, ask yourself:

- **What exactly is being asked?** (Not what you think is being asked.)
- **What information do I have?** (The inputs.)
- **What result do I need?** (The output.)
- **Are there any constraints?** (Rules, limits, special conditions.)

Example:

Problem: "Write a program to find the largest number in a list."

Before writing anything, think:

- Input: A list of numbers. How many numbers? Can they be negative?
- Output: One number — the largest one.
- Constraint: What if the list is empty?

Understanding the problem means answering these questions first.

Interactive Stop-Point: Pause & Think

Read this problem statement:

"A school canteen wants to know how many students bought lunch today. The canteen serves from 12:00 PM to 1:00 PM."

Answer these questions:

1. What is the **input** to this problem?
 2. What is the **output** we need?
 3. What **constraint** exists? (Look at the time.)
 4. What information is missing that you would need to ask about?
-

Quick Recap

Understand the problem fully — its inputs, outputs, and constraints — before writing a single line of solution.

4.2.2 Problem Simplification

The Hook

Think of a jigsaw puzzle with 1,000 pieces. You cannot solve it all at once. What do you do? You sort pieces by color. You find the corner pieces first. You build one section at a time.

Problem simplification is the same idea: **divide the big problem into smaller sub-problems, and solve each one.**

The Explanation

Definition: Problem Simplification means breaking a complex problem into smaller, more manageable sub-problems that are easier to understand and solve.

This principle connects directly to **decomposition** (Section 4.1.1). But here, we apply it specifically as a step in the problem-solving process.

Example: Designing a Website

Sub-Problem	What It Involves
1. Design the layout	How many pages? Where does the menu go?
2. Create the content	Write the text. Choose the images.
3. Code the functionality	Make buttons work. Add forms.
4. Test the website	Check for broken links. Test on different devices.
5. Launch the website	Upload to a server.

Each sub-problem can be worked on separately — even by different team members. This is how professional software is built.

Interactive Stop-Point: Grab a Partner

Simplify this problem into at least 4 sub-problems:

"Plan a school sports day event for 200 students."

Write each sub-problem as a separate task. Then, compare your list with another pair. Did you miss anything they included?

Quick Recap

Simplify any large problem by dividing it into smaller sub-problems. Solve each piece. The full solution follows.

4.2.3 Solution Selection and Design

The Hook

You need to travel from Lahore to Karachi. You have three options: plane, train, or bus. Each one is a valid solution. But which is the **best** solution for your situation?

- If you have very little time: choose the plane.
- If you have little money: choose the bus.
- If you want comfort at a reasonable price: choose the train.

Choosing the best solution requires **evaluation**. The same is true in computer science.

The Explanation

Definition: Solution Selection and Design means evaluating different possible approaches to a problem and choosing the most efficient one — then creating a detailed plan (an algorithm) to implement it.

How to evaluate solutions:

Criteria	Question to Ask
Correctness	Does it actually solve the problem?
Efficiency	Is it fast? Does it use less memory?
Simplicity	Is it easy to understand and maintain?
Scalability	Will it still work if the problem gets bigger?

Once you select the best solution, you **design** it — meaning you write out the algorithm, create a flowchart, or write pseudocode to represent it clearly.

Interactive Stop-Point: Pause & Think

You want to find a specific name in a list of 100 student names. Here are two approaches:

- **Approach A:** Read every name from the beginning until you find it.
- **Approach B:** Sort the list alphabetically first, then jump directly to the right section (like a dictionary).

Which approach is better? Why? What factors does your choice depend on?

Quick Recap

Evaluate your options. Choose the solution that is correct, efficient, and practical. Then design it clearly.

4.3 Algorithm Design Methods

Introduction

You have an algorithm in your head. Now how do you **communicate** it to someone else — or to a computer?

There are two main methods:

1. **Flowcharts** — visual diagrams
2. **Pseudocode** — structured plain-language text

Both methods describe the same algorithm. They just do it in different ways. Let's learn both.

4.3.1 Flowcharts

The Hook

In the 1940s, **Herman Goldstine** and **John von Neumann** — two pioneering computer scientists — needed a way to explain complex computer programs to their teams. Writing out long paragraphs of instructions was confusing. So they invented a system of **boxes and arrows** that showed the flow of logic visually.

That system became the **flowchart**. Today, it is used by computer scientists, engineers, business analysts, and students around the world. You are about to join that tradition.

Importance of Flowcharts

Definition: A flowchart is a visual representation of the steps in a process or algorithm, using different symbols connected by arrows.

Why are flowcharts important?

Benefit	Explanation
Clarity	A picture shows the full process at a glance.
Communication	Anyone — even without coding knowledge — can read a flowchart.
Problem Solving	Flowcharts help you spot steps that are missing or incorrect.
Documentation	They serve as a permanent record of how a system works.

Flowchart Symbols

Every flowchart uses a small set of standard symbols. Learn these five — they are all you need.

Symbol Shape	Name	What It Represents
Oval	Terminal	The Start or End of the process. Always labeled "Start" or "End/Stop".
Rectangle	Process	A task or operation being performed. Example: "Calculate Sum = A + B"
Parallelogram	Input / Output	Reading input from the user OR displaying output . Example: "Read A" or "Display Sum"
Diamond	Decision	A yes/no question or condition . The flow branches into two paths based on the answer.
Arrow	Flowline	Shows the direction of flow — which step comes next.

How to draw these symbols on paper:

- Oval: Draw an elongated circle (like a running track shape).
- Rectangle: A standard rectangle box.
- Parallelogram: A slanted rectangle (leaning to the right).
- Diamond: A square rotated 45°, like a diamond shape.
- Arrow: A line with an arrowhead at the end.

Flowchart Examples

Example 1: Adding Two Numbers

Here is how to describe this flowchart (draw it as you read):

```

[START] – oval
↓
[READ A] – parallelogram (input)
↓
[READ B] – parallelogram (input)
↓
[Sum = A + B] – rectangle (process)
↓
[DISPLAY Sum] – parallelogram (output)
↓
[STOP] – oval

```

Simple. Clean. Six steps. Anyone can follow it.

Example 2: Finding If a Number Is Even or Odd

```

[START] – oval
↓
[READ No.] – parallelogram (input)
↓
[Is No. % 2 == 0 ?] – diamond (decision)
  ↙ YES                ↘ NO
[DISPLAY "Even"]      [DISPLAY "Odd"]
– parallelogram        – parallelogram
  ↓                    ↓
                [STOP] – oval

```

The **diamond** creates a fork in the road. If the remainder when divided by 2 is zero, the number is even. Otherwise, it is odd.

Example 3: Login System (Maximum 5 Attempts)

```

[START]
↓
[SET Attempts = 0]
↓
[READ Username and Password]
↓
[Are credentials correct?] – diamond
  ↙ YES                ↘ NO
[GRANT ACCESS]         [Attempts = Attempts + 1]
[STOP]                ↓
                        [Is Attempts == 5?] – diamond
                        ↙ YES                ↘ NO
                        [LOCK ACCOUNT]      [GO BACK to READ Username and Password]
                        [ALERT USER]
                        [STOP]

```

This flowchart shows **looping** (going back to try again) and **multiple decision points**.

Class Activity

Draw a flowchart for selecting the school cricket team.

Rules:

- The team can have a maximum of **11 players**.
- Each player must have **parental permission**.

Start with: "Is the player's permission form signed?" and "Is the team full (11 players)?"

Interactive Stop-Point: Pause & Think

A vending machine gives you a snack if you insert enough money. If you insert too little, it asks for more. If the snack is out of stock, it returns your money.

Can you **describe** this as a flowchart in words? Write out each step and decision. Then try to draw it.

Quick Recap

A flowchart uses standard symbols (oval, rectangle, parallelogram, diamond, arrow) to visually show the steps and decisions in an algorithm.

4.3.2 Pseudocode

The Hook

Imagine you are writing a recipe — but not for a person. You are writing it for a robot that can read English but needs very precise, structured instructions. You cannot use casual language like "add a little salt." You must write "Add 1 teaspoon of salt." But you also do not need to write actual code.

That middle ground — **structured, precise, plain language** — is pseudocode.

The Explanation

Definition: Pseudocode is a method of representing an algorithm using simple, structured, informal language that is easy for humans to read. It combines the structure of programming with the clarity of plain English.

Important: Pseudocode is NOT actual code. It cannot be run on a computer. It is a **planning tool**. You use it to think through your logic before writing real code.

Why use pseudocode?

Reason	Explanation
Clarity	Focus on logic, not syntax rules.
Planning	Outline your algorithm before coding.
Communication	Share ideas with teammates who use different coding languages.
Language-independent	Not tied to Python, Java, or any specific language.

Pseudocode Examples

Example 1: Checking If a Number Is Even or Odd

```

Procedure CheckEvenOdd(number)
  Input:  number  (the number to check)
  Output: "Even" or "Odd"

  BEGIN
    READ No.
    IF (No. % 2 == 0) THEN
      PRINT "No. is Even"
    ELSE
      PRINT "No. is Odd"
    END IF
  END
END

```

Line-by-line explanation:

- **Procedure** — this is the name of our algorithm.
 - **READ No.** — we get the number from the user.
 - **IF (No. % 2 == 0)** — we check if the remainder after dividing by 2 is zero.
 - **THEN PRINT "Even"** — if yes, print Even.
 - **ELSE PRINT "Odd"** — if no, print Odd.
 - **END IF** — closes the decision block.
 - **END** — the algorithm is complete.
-

Example 2: Checking If a Number Is Prime

A **prime number** is a number greater than 1 that has no divisors other than 1 and itself.
(Examples: 2, 3, 5, 7, 11, 13...)

```

Procedure IsPrime(number)
  Input:  number  (the number to check)
  Output: True (prime) or False (not prime)

  BEGIN
    IF (number <= 1) THEN
      RETURN False
    END IF

    FOR i FROM 2 TO sqrt(number) DO
      IF (number % i == 0) THEN
        RETURN False
      END IF
    END FOR

    RETURN True
  END
END

```

Key ideas:

- We only check divisors up to the **square root** of the number. This is more efficient.
 - If any divisor divides the number evenly (remainder = 0), it is NOT prime.
 - If no divisor divides it, it IS prime.
-

Example 3: Login System — Username and Password Check

```
Procedure CheckCredentials(username, password)
  Input:  username, password
  Output: Login success or failure message

  BEGIN
    validUsername = "user123"
    validPassword = "pass123"

    IF (username == validUsername) THEN
      IF (password == validPassword) THEN
        PRINT "Login successful"
      ELSE
        PRINT "Invalid password"
      END IF
    ELSE
      PRINT "Invalid username"
    END IF
  END
```

Notice the **nested IF** — one decision inside another. This is common in real programs.

Class Activity

Divide into small groups. Each group gets one of these problems:

1. Find the **maximum number** in a list of five numbers.
2. Calculate the **factorial** of a number. (Factorial of 5 = $5 \times 4 \times 3 \times 2 \times 1 = 120$)
3. Check if a student **passed or failed** (passing mark = 50).

Write pseudocode for your problem. Present it to the class. Can the class follow your logic?

Interactive Stop-Point: Pause & Think

Here is a piece of pseudocode:

```
BEGIN
  READ score
  IF score >= 80 THEN
    PRINT "Grade: A"
  ELSE IF score >= 60 THEN
    PRINT "Grade: B"
  ELSE IF score >= 40 THEN
    PRINT "Grade: C"
  ELSE
    PRINT "Grade: Fail"
  END IF
END
```

What grade would be printed for the following scores?

1. Score = 75
2. Score = 92
3. Score = 38
4. Score = 60

Quick Recap

Pseudocode uses structured, plain-language instructions to describe an algorithm clearly — without using any specific programming language syntax.

4.3.3 Differentiating Flowcharts and Pseudocode

Both flowcharts and pseudocode describe the **same algorithm** — but in very different ways. The table below summarizes the key differences:

Feature	Flowchart	Pseudocode
Format	Visual (shapes and arrows)	Text (structured plain language)
Reading style	Like watching a map — see the whole picture at once	Like reading a story — step by step
Best for	Understanding the flow and structure of an algorithm	Planning and converting directly to code
Audience	Anyone — including non-programmers	Programmers and technical students
Decisions	Shown as diamond shapes with branches	Shown as IF...THEN...ELSE blocks
Ease of change	Harder to edit once drawn	Easy to revise — just rewrite the line

Which should you use?

- Use a **flowchart** when you want to visualize the process or present it to an audience.
- Use **pseudocode** when you want to plan logic before writing actual code.
- Many professionals use **both** — a flowchart for the big picture, pseudocode for the detail.

Interactive Stop-Point: Grab a Partner

Your partner describes an algorithm verbally: *"Ask the user for their age. If the age is 18 or above, print 'You can vote.' Otherwise, print 'You cannot vote yet.'"*

You draw the **flowchart**. **Your partner** writes the **pseudocode**.

Compare your results. Do they represent the same logic?

Quick Recap

Flowcharts show the visual flow. Pseudocode shows the structured logic. Both describe the same algorithm in different formats.

4.4 Evaluation Techniques for an Algorithm

Introduction

You have designed an algorithm. It works. But is it **good**? Could a different algorithm solve the same problem **faster** or using **less memory**?

Evaluating an algorithm means measuring how well it performs. There are two key measurements:

1. **Time Complexity** — how fast is it?
 2. **Space Complexity** — how much memory does it use?
-

4.4.1 Time Complexity

The Hook

Imagine you are searching for your friend's name in a school attendance register. The register has 30 names. You start from the top and read each name until you find your friend. If your friend's name starts with "Z", you might have to read all 30 names.

Now imagine the register has **1,000 names**. Or **10,000 names**.

The more names, the more time it takes. **Time complexity** is the study of exactly how that time grows as the input gets larger.

The Explanation

Definition: Time complexity is a measure of how the running time of an algorithm changes as the size of the input increases.

We usually express time complexity using **Big O notation**. You do not need to calculate it mathematically at this level — just understand what it means.

Big O Notation	Name	What It Means
O(1)	Constant Time	The algorithm always takes the same time, regardless of input size. Example: looking up a value in a known location.
O(n)	Linear Time	As input doubles, time doubles. Example: searching through a list one by one.
O(n²)	Quadratic Time	As input doubles, time quadruples. Example: comparing every item with every other item.
O(log n)	Logarithmic Time	As input doubles, time increases only slightly. Example: binary search on a sorted list.

Simple analogy:

- You have a box of 100 chocolates. Finding a specific chocolate by checking one by one = **O(n)**.
- If the chocolates are sorted by flavor and you split the box in half each time = **O(log n)**.
- O(log n) is much faster. This is why sorting data before searching it is often worth the effort.

Class Activity

Think of a simple task: finding the **largest number** in a list.

Write down the steps. Now imagine the list has:

- 10 numbers
- 100 numbers
- 1,000 numbers

How do the steps change? Does the number of comparisons increase? By how much?

Interactive Stop-Point: Pause & Think

Which algorithm do you think is faster for finding a name in a list of 1,000 names?

- **Algorithm A:** Start at the beginning. Check each name. Stop when you find it.
- **Algorithm B:** The list is sorted alphabetically. Guess the middle. If your name comes before the middle, discard the second half. Repeat.

Why is Algorithm B faster? What does this tell you about the relationship between **data organization** and **time complexity**?

Quick Recap

Time complexity tells us how an algorithm's running time grows as the input size grows. Faster algorithms have lower time complexity.

4.4.2 Space Complexity

The Hook

You are packing for a camping trip. One of your friends brings a single backpack with everything neatly organized. Another friend brings three large suitcases for the same trip.

Both friends are prepared. But your organized friend is much more **efficient** in their use of space.

Algorithms work the same way. Two algorithms might both solve a problem correctly — but one might use far more **memory** to do it.

The Explanation

Definition: Space complexity is a measure of how much memory an algorithm needs relative to the size of its input.

Space complexity includes:

- **Input space:** The memory needed to store the input data.
- **Auxiliary space:** Extra memory the algorithm needs while running (temporary variables, lists, etc.).

Example:

An algorithm that sorts a list by copying it into a second list uses **more memory** than one that sorts it **in place** (rearranging items within the same list). The in-place algorithm has lower space complexity.

Like time complexity, space complexity is also expressed in Big O notation.

The trade-off: Sometimes you can make an algorithm faster by using more memory. Other times, saving memory makes the algorithm slower. A good computer scientist understands this trade-off.

Interactive Stop-Point: Pause & Think

You have a list of 10 numbers. You want to find the **average**.

- **Method A:** Add all numbers one by one, keeping a running total. Use only 2 extra variables (total and count).
- **Method B:** Copy all 10 numbers into a second list, then add them.

Which method uses **less space**? Which would you prefer if memory was limited? Does the choice change if the list has 1 million numbers?

Quick Recap

Space complexity measures how much memory an algorithm uses. Good algorithms are efficient in both time and space.

4.5 Dry Run

Introduction

You have written an algorithm or drawn a flowchart. How do you know it is correct **before** running it on a computer?

The answer: you **dry run** it. A dry run means you trace through the algorithm **manually**, step by step, using sample data, with a pen and paper. You track the value of each variable at each step.

A dry run catches logical errors early — saving time and frustration later.

4.5.1 Dry Run of a Flowchart

The Hook

Before a pilot flies a new aircraft, they sit in a simulator and "fly" the plane without leaving the ground. They go through every step — engine startup, takeoff, navigation, landing — to make sure they know exactly what to do and that everything works.

A dry run is the same idea. You "fly" your flowchart without a computer.

Step-by-Step Walkthrough

Flowchart: Adding Two Numbers

Flowchart steps (as described in Section 4.3.1):

```
[START] → [Input A] → [Input B] → [Sum = A + B] → [Display Sum] → [STOP]
```

Dry Run with sample values: A = 3, B = 5

Step	Action	Variable Values
1	START	—
2	Input A	A = 3
3	Input B	B = 5
4	Sum = A + B	Sum = 3 + 5 = 8
5	Display Sum	Output: 8
6	STOP	—

The algorithm correctly outputs 8. The dry run confirms the flowchart is correct.

Class Activity

Draw a flowchart for finding the **largest of two numbers**. Then perform a dry run for:

- Numbers 7 and 4
- Numbers 12 and 19

Write down each step and the value of all variables at each step. What does your flowchart output? Is it correct?

Interactive Stop-Point: Pause & Think

A student drew this flowchart for finding if a number is positive or negative. During the dry run with the number **0**, the flowchart displayed "Positive."

Is that correct? What change would you make to the flowchart to handle the number 0 properly?

Quick Recap

A dry run of a flowchart means manually tracing through each step with sample data, tracking variables, to verify the logic is correct.

4.5.2 Dry Run of Pseudocode

The Hook

Imagine you are a teacher marking a student's pseudocode. You go through it line by line, mentally running the instructions with a specific example, checking whether the output makes sense. That is exactly what a dry run of pseudocode is.

Step-by-Step Walkthrough

Pseudocode: Finding the Maximum of Two Numbers

```
Algorithm FindMax
  Input: num1, num2

  IF num1 > num2 THEN
    max = num1
  ELSE
    max = num2
  END IF

  OUTPUT max
```

Dry Run with values: num1 = 10, num2 = 15

Line	Instruction	Variable Values	Notes
1	Input num1 = 10, num2 = 15	num1=10, num2=15	
2	IF 10 > 15 THEN	—	Condition is FALSE
3	ELSE	—	Go to ELSE branch
4	max = num2	max = 15	
5	END IF	—	
6	OUTPUT max	Output: 15	Correct!

The algorithm correctly identifies 15 as the maximum.

Now try with: **num1 = 25, num2 = 10**

Line	Instruction	Variable Values	Notes
1	Input num1 = 25, num2 = 10	num1=25, num2=10	
2	IF 25 > 10 THEN	—	Condition is TRUE
3	max = num1	max = 25	
4	END IF	—	
5	OUTPUT max	Output: 25	Correct!

Interactive Stop-Point: Grab a Partner

Dry run the following pseudocode with the values **num1 = 7, num2 = 7** (both equal):

```
IF num1 > num2 THEN
    max = num1
ELSE
    max = num2
END IF
OUTPUT max
```

- What does the output?
- Is the output correct (is 7 the maximum of 7 and 7)?
- What would happen if we changed **>** to **>=**? Would the output change?

Quick Recap

A dry run of pseudocode means tracing through each line manually with sample values to verify that the logic and output are correct.

4.5.3 Simulation

The Hook

In 2003, NASA sent two rovers — Spirit and Opportunity — to the surface of Mars. Before launching them into space, NASA engineers ran **thousands of computer simulations** to test what would happen when they landed, how the terrain would affect the wheels, and what could go wrong.

They could not test on Mars. So they built a digital model of Mars and tested there first.

That is simulation: using a computer model to imitate a real-world system.

The Explanation

Definition: Simulation is the use of computer programs to create a model of a real-world process or system, allowing us to test ideas and algorithms without real-world risk or cost.

Why use simulation?

Benefit	Example
Cost-effective	Testing aircraft designs digitally costs less than building real prototypes.
Safe	Simulating a building fire to test evacuation plans — without actually burning a building.
Repeatable	Run the same simulation 1,000 times with different settings.
Exploratory	Try scenarios that haven't happened yet.

Real-world examples:

- **Weather forecasting:** Meteorologists input temperature, humidity, and wind data into computer models to predict tomorrow's weather.
 - **Traffic flow:** City planners simulate how changing a traffic light timing affects congestion.
 - **Medical training:** Doctors practice surgery on digital simulations before performing on real patients.
 - **Video games:** Physics engines simulate gravity, collision, and movement in game worlds.
-

Class Activity: Simulation Game

Objective: Experience managing a system and making decisions.

Required materials: A computer or tablet with internet access; a city-building simulation game (e.g., SimCity or similar).

Activity: Play in pairs. Build and manage a virtual city. Make decisions about roads, power, water, and buildings. After 20 minutes, discuss:

- What decisions were hardest?
- What happened when you ran out of resources?
- How did changing one thing (a road, a power plant) affect the whole city?

Link to learning: Real cities use simulations to plan. You just did the same thing.

Interactive Stop-Point: Pause & Think

A hospital wants to know if it has enough doctors to handle an outbreak of flu during the winter. They cannot wait for winter to find out.

How could they use **simulation** to plan ahead? What information (inputs) would they need to include in the simulation? What outputs would they want?

Quick Recap

Simulation creates a digital model of a real-world system, allowing safe, repeatable, and cost-effective testing of algorithms and plans.

4.6 Introduction to LARP (Logic of Algorithms for Resolution of Problems)

What Is LARP?

Definition: LARP stands for **Logic of Algorithms for Resolution of Problems**. It is a software tool that allows students to **write, draw, and execute algorithms** in a simplified, beginner-friendly environment.

Think of LARP as a **playground for algorithms**. You write your steps, run them, and see the results immediately. You can experiment, make mistakes, and learn — all in a safe, visual environment.

4.6.1 Why Is LARP Important?

The Hook

Learning to play guitar is much more effective if you actually **pick up the guitar** and play, rather than just reading a book about music theory. LARP is the guitar for algorithm learners. It puts the algorithm in your hands.

The Explanation

LARP helps you:

1. **See** how algorithms execute step by step.
2. **Understand** the effect of different inputs on the output.
3. **Practice** writing algorithms in a safe, interactive environment.
4. **Visualize** flowcharts and immediately run them to check their correctness.
5. **Debug** errors using real error messages.

LARP makes the abstract concept of an algorithm **concrete and real**. Instead of imagining what happens when you run an algorithm, you actually run it and watch.

Interactive Stop-Point: Pause & Think

Before using LARP, think about: what is the difference between **writing** an algorithm and **running** it? What can running reveal that writing alone cannot?

Quick Recap

LARP is a tool that lets you write, draw, and run algorithms interactively — turning theory into hands-on practice.

4.6.2 Writing Algorithms in LARP

LARP Syntax — The Commands

LARP uses a simple, structured language. Here are the key commands:

Command	Purpose	Example
START	Begin the algorithm	START
END	Finish the algorithm	END
WRITE	Display a message or value	WRITE "Enter your salary"
READ	Take input from the user	READ salary
IF...THEN...ELSE...ENDIF	Make a decision	IF salary > 50000 THEN
FOR...DO...ENDFOR	Repeat a block of steps	FOR i FROM 1 TO 10 DO
WHILE...DO...ENDWHILE	Repeat while a condition is true	WHILE answer != "quit" DO

LARP Algorithm Examples

Example 1: Check If a Number Is Even or Odd

```

START
  WRITE "Enter a number"
  READ number
  IF number % 2 == 0 THEN
    WRITE "The number is even"
  ELSE
    WRITE "The number is odd"
  ENDIF
END

```

What happens when you run this in LARP:

1. LARP displays: *Enter a number*
2. You type: 6
3. LARP checks: $6 \% 2 = 0 \rightarrow$ Condition is TRUE
4. LARP displays: *The number is even*

Example 2: Calculate Tax on Annual Salary

```
START
  WRITE "Enter monthly salary"
  READ salary
  annualSalary = salary * 12
  WRITE "Annual Salary: "
  WRITE annualSalary
  IF annualSalary > 600000 THEN
    WRITE "Tax applies"
  ELSE
    WRITE "No tax"
  ENDIF
END
```

Interactive Stop-Point: Grab a Partner

Write a LARP algorithm that:

1. Asks the user for a student's marks (out of 100).
2. If marks are 50 or above, display "PASS".
3. If marks are below 50, display "FAIL".

Write it using LARP syntax. Then swap with your partner and check each other's work. Are the START and END commands present? Is the IF...THEN...ELSE...ENDIF structure correct?

Quick Recap

LARP uses simple commands — START, END, WRITE, READ, IF...THEN...ELSE — to write and execute algorithms interactively.

4.6.3 Drawing Flowcharts in LARP

The Explanation

LARP also has a **visual flowchart mode**. In this mode, you drag and drop standard flowchart shapes — ovals, rectangles, parallelograms, diamonds — onto a canvas and connect them with arrows.

Once your flowchart is complete, LARP can **execute it** — simulating the flow of control through each shape and displaying the output.

Standard shapes in LARP's flowchart mode:

Shape	Name	Use in LARP
Oval	Terminal	START and STOP nodes
Rectangle	Process	Calculations, assignments
Parallelogram	Input/Output	READ (input) and WRITE (output)
Diamond	Decision	IF conditions — two branches: YES and NO
Arrow	Flowline	Connects shapes, shows direction

Example: Flowchart for Checking a Student's Grade in LARP

```

[START]
  ↓
[READ percentage] – parallelogram
  ↓
[Is percentage >= 60?] – diamond
  ✓ YES                ↘ NO
[WRITE "Grade A"]    [WRITE "Below A"]
– parallelogram      – parallelogram
  ↓                  ↓
                [STOP]

```

After drawing this in LARP, you click **Run**. LARP asks you to enter a percentage. You type 75. LARP follows the YES branch and displays "Grade A."

This visual feedback is what makes LARP powerful for learners.

Class Activity

Open LARP. Draw a flowchart that:

1. Asks the user to enter an integer.
2. Checks if the number is greater than 100.
3. If YES, displays "Large number."
4. If NO, displays "Small number."

Run the flowchart with these test values: 50, 100, 150. Record your outputs. Are they correct?

Interactive Stop-Point: Pause & Think

A student draws a flowchart in LARP where the **arrow from the diamond goes back to the READ step**. What does this create in the algorithm? Is that useful? Can you think of a situation where going back (looping) is necessary?

Quick Recap

LARP's flowchart mode lets you visually build and execute flowcharts using standard shapes, giving immediate feedback on your algorithm's logic.

4.7 Error Identification and Debugging

Introduction

Here is something very important to know:

Every programmer makes mistakes. Every single one. From beginners to experts.

The difference between a good programmer and a struggling one is not that the good programmer never makes errors. It is that the good programmer knows **how to find and fix them**.

Errors in an algorithm are called **bugs**. The process of finding and fixing bugs is called **debugging**.

Fun Fact: The term "debugging" comes from 1947, when engineers found an actual moth stuck inside a computer at Harvard University. The moth was causing the machine to malfunction. They removed it — literally "debugging" the computer. The logbook entry still exists today.

4.7.1 Types of Errors

There are three main types of errors you will encounter:

Error Type 1: Syntax Errors

Definition: A syntax error occurs when the algorithm breaks the rules of the language — like using the wrong command, misspelling a keyword, or forgetting a required word.

Analogy: It is like writing in English but saying "The ball the hit boy" instead of "The boy hit the ball." The grammar is wrong.

LARP Example:

```

START
  WRITE "Enter number"
  READ num
  IF num > 0
    WRITE "Positive"
  ENDIF
END

```

Error: The **IF** statement is missing **THEN**. The correct line is:

```
IF num > 0 THEN
```

How to spot it: LARP will usually **underline or highlight** the incorrect line and show an error message telling you exactly where the problem is.

Key insight: Syntax errors are the **easiest** to fix because the tool tells you where they are.

Error Type 2: Runtime Errors

Definition: A runtime error occurs while the algorithm is running — it happens because the algorithm tries to do something **impossible**.

Analogy: The grammar of your sentence is fine, but you are asking someone to "divide a pizza into zero slices." That is impossible at the moment of execution.

Common runtime errors:

- **Dividing by zero:** `result = 10 / 0` — mathematically undefined.
- **Using a variable before giving it a value:** `WRITE total` when `total` was never calculated.
- **Going beyond the end of a list:** Asking for item 15 in a list that only has 10 items.

LARP Example:

```

START
  WRITE "Enter denominator"
  READ denom
  result = 100 / denom
  WRITE result
END

```

If the user enters **0**, LARP will throw a **runtime error** (division by zero) and stop execution.

Fix: Add a check before the division:

```
IF denom == 0 THEN
    WRITE "Error: Cannot divide by zero"
ELSE
    result = 100 / denom
    WRITE result
END IF
```

Error Type 3: Logical Errors

Definition: A logical error is when the algorithm runs without crashing, but produces the **wrong output** because the logic is incorrect.

Analogy: Your sentence is grammatically correct. It is possible. But it gives the wrong answer. Like saying "5 + 3 = 9."

This is the hardest type of error to find — because the computer does not tell you about it. The algorithm runs fine. But the output is wrong.

Example:

A student wants to find the average of three numbers. They write:

```
START
    READ a
    READ b
    READ c
    average = a + b + c
    WRITE average
END
```

The error: The student forgot to **divide by 3**. The algorithm outputs the sum, not the average. No error message appears. But the answer is wrong.

Correct version:

```
average = (a + b + c) / 3
```

How to find logical errors: Dry runs. Trace through your algorithm manually with known values. If the output from the dry run does not match your expected answer, you have a logical error.

Summary Table: Types of Errors

Type	When It Occurs	How It's Detected	Difficulty to Fix
Syntax	When you write the algorithm	LARP highlights it immediately	Easy
Runtime	When the algorithm is running	LARP stops and shows a message	Medium
Logical	When the algorithm runs AND gives wrong output	You must compare output to expected results	Hard

Class Activity

Create a simple LARP flowchart that calculates the average of three numbers.

Then deliberately introduce:

1. A **syntax error** (e.g., remove a required keyword).
2. A **runtime error** (e.g., allow division by zero).
3. A **logical error** (e.g., divide by 2 instead of 3).

Run each version. Record the error messages or wrong outputs. Then fix all three errors.

Interactive Stop-Point: Pause & Think

Look at this pseudocode:

```
START
  READ temperature
  IF temperature > 100 THEN
    WRITE "Water is boiling"
  IF temperature < 0 THEN
    WRITE "Water is freezing"
  ELSE
    WRITE "Water is liquid"
  ENDIF
END
```

1. Is there a **syntax error**? Look carefully at the structure.
2. If temperature = 110, what will be displayed? Is that correct?
3. What type of error is present?

Quick Recap

There are three types of errors: syntax (wrong structure), runtime (impossible operation), and logical (wrong output). Debugging means finding and fixing all three.

4.7.2 Common Error Messages in LARP

When LARP encounters a problem, it shows an **error message**. These messages are clues — not insults. Learn to read them carefully.

Error Message	What It Means	What to Do
Missing Step	You forgot an important step or command (e.g., missing <code>THEN</code> , <code>ENDIF</code> , or <code>END</code>).	Review the flagged line. Check that all blocks are properly opened and closed.
Undefined Variable	You are using a variable (like <code>total</code>) that was never given a value using <code>READ</code> or an assignment.	Add a <code>READ total</code> or <code>total = 0</code> before using it.
Invalid Operation	You are attempting something impossible — like dividing by zero or using a text value where a number is needed.	Add an <code>IF</code> check to prevent the impossible situation.
Unexpected End	The algorithm ended before all blocks were closed — a missing <code>ENDIF</code> , <code>ENDFOR</code> , or <code>END</code> .	Count your opening and closing statements. Every <code>IF</code> needs an <code>ENDIF</code> .

Debugging Strategy: The 4-Step Method

When you find a bug in your algorithm, follow these steps:

1. **Read the error message carefully.** What is it telling you?
2. **Find the location.** LARP usually highlights the line with the error.
3. **Understand the cause.** Why did this error happen? (Wrong syntax? Impossible operation? Wrong logic?)
4. **Fix and re-run.** Make the change. Run the algorithm again with the same test data.

If no error message appears but the output is wrong, do a **dry run** — trace through the algorithm manually.

Interactive Stop-Point: Grab a Partner

Here is a buggy LARP algorithm. It is supposed to print all numbers from 1 to 5:

```
START
  SET i = 1
  WHILE i <= 5 DO
    WRITE i
  ENDWHILE
END
```

Spot the bug: What is missing? What will happen when you run this? What type of error is it?

(Hint: What happens to the value of `i` inside the loop?)

Quick Recap

LARP error messages are clues. Read them carefully, find the cause, fix it, and re-run. Always test with multiple values to confirm the fix works.

Chapter Summary

This chapter introduced you to one of the most important skills in computer science — and in life: **computational thinking**.

Here is everything you learned:

Topic	Key Takeaway
Computational Thinking	A problem-solving approach using decomposition, pattern recognition, abstraction, and algorithms.
Decomposition	Break big problems into smaller, manageable pieces.
Pattern Recognition	Find repeating patterns to predict and reuse solutions.
Abstraction	Focus on what matters; ignore unnecessary details.
Algorithms	A clear, ordered, finite set of steps to solve a problem.
Problem Understanding	Fully analyze the problem — inputs, outputs, and constraints — before solving.
Problem Simplification	Divide complex problems into sub-problems.
Solution Design	Evaluate options; choose the most efficient approach; plan it clearly.
Flowcharts	Visual diagrams using ovals, rectangles, parallelograms, diamonds, and arrows.
Pseudocode	Structured plain-language description of an algorithm.
Time Complexity	How running time grows as input size grows.
Space Complexity	How much memory an algorithm needs.
Dry Run	Manually trace through an algorithm with sample data to verify correctness.
Simulation	Use computer models to test algorithms and systems safely.
LARP	A tool for writing, drawing, and running algorithms interactively.
Syntax Errors	Wrong structure or spelling in the algorithm — easiest to detect.
Runtime Errors	Impossible operations detected during execution.
Logical Errors	Wrong output despite correct structure — hardest to detect.

Key Vocabulary

Term	Definition
Algorithm	A step-by-step set of instructions to solve a problem.
Abstraction	Hiding unnecessary details to focus on what is important.
Big O Notation	A way to express time or space complexity (e.g., $O(n)$, $O(\log n)$).
Bug	An error in an algorithm or program.
Computational Thinking	A structured, logical approach to problem-solving.
Debugging	The process of finding and fixing errors.
Decomposition	Breaking a problem into smaller, manageable sub-problems.
Dry Run	Manually tracing through an algorithm with sample data.
Flowchart	A visual diagram representing the steps and decisions in an algorithm.
LARP	Logic of Algorithms for Resolution of Problems — a learning tool for algorithms.
Pattern Recognition	Identifying regularities or repeating structures in data or problems.
Pseudocode	Structured, plain-language description of an algorithm.
Simulation	A computer model of a real-world process used for testing.
Space Complexity	A measure of how much memory an algorithm uses.
Syntax Error	An error caused by incorrect structure or spelling in an algorithm.
Time Complexity	A measure of how running time grows with input size.

Review Questions

Section 4.1 — Computational Thinking

1. Define computational thinking in your own words.
2. What is decomposition? Give one example from daily life.
3. In the square area pattern (1, 4, 9, 16, 25...), what is the rule for the pattern?
4. What does abstraction mean? Why is it useful in problem-solving?
5. List four properties of a good algorithm.

Section 4.2 — Principles

1. Why is understanding the problem the most important step?
2. Give an example of problem simplification from your school life.
3. What four criteria would you use to choose the best solution to a problem?

Section 4.3 — Algorithm Design Methods

1. Name and describe the five standard flowchart symbols.
2. Draw a flowchart for checking if a student passed (passing marks = 50).
3. What is pseudocode? How is it different from actual code?
4. Write pseudocode to print the numbers 1 to 10.
5. Give two situations where you would prefer a flowchart over pseudocode, and two where you would prefer pseudocode.

Section 4.4 — Evaluation Techniques

1. What does $O(n)$ mean in simple terms?
2. Describe one situation where space complexity matters more than time complexity.

Section 4.5 — Dry Run

1. Why is a dry run useful before running an algorithm on a computer?
2. Perform a dry run of the `FindMax` pseudocode (Section 4.5.2) for `num1 = 3` and `num2 = 8`. Show each step.
3. What is simulation? Give two real-world examples of simulation.

Section 4.6 — LARP

1. What does LARP stand for? What is its purpose?
2. Write a LARP algorithm that asks for a temperature and displays "Hot" if it is above 35, "Cold" if it is below 10, and "Comfortable" otherwise.
3. What is the difference between the `WRITE` and `READ` commands in LARP?

Section 4.7 — Error Identification and Debugging

1. What is the difference between a syntax error and a logical error?
2. Give an example of a runtime error. How would you fix it?
3. A program calculates the average of two numbers but always gives the wrong answer. What type of error is this likely to be? How would you find it?
4. List three common LARP error messages and what they mean.

End of Unit 4: Computational Thinking

Remember: You are not just learning computer science. You are learning how to think — how to break problems apart, spot patterns, ignore the noise, and build solutions step by step. Those skills will serve you in every subject, every career, and every challenge you face. You have got this.